| Ex.No.1B | CODE GENERATION WITH TEST CASES | URK23CS8018 |
|---|---|---|
| 20-08-2026 | | |

**Aim:**

To generate a Python function from a natural language description using Ollama and test it with multiple test cases.

**Procedure:**

**Step 1:** Import the Ollama library.
**Step 2:** Provide a natural language description of the required function.
**Step 3:** Generate the Python function using the Llama3 model.
**Step 4:** Execute the generated function dynamically using exec().
**Step 5:** Test the function using multiple test cases.
**Step 6:** Compare the output with the expected result.
**Step 7:** Display whether each test case passes or fails.

**Question:**

Code Generation with Test Cases
Generate a Python function named n_nat_numbers that returns numbers from 0 to a given number as a list and test it using multiple test cases.

**Program:**

```
import ollama
import traceback

def generate_function(description):
    prompt = f"""Generate a Python function based on the following description:

Description:
{description}

Make sure the function is valid and contains no extra print statements or explanations.
Return only the function code.
"""

    response = ollama.chat(
        model="llama3:8b",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
)
```

```
return response["message"]["content"]
def execute_function(function_code, function_name, test_cases):
    try:
        # Define the function dynamically
        local_env = {}
        # Remove markdown code blocks if the model generates them
        function_code = function_code.replace("```python", "").replace("```", "").strip()
        exec(function_code, globals(), local_env)
        func = local_env[function_name]
        results = []
        for inputs, expected in test_cases:
            try:
                output = func(*inputs)
                results.append(
                    (inputs, output, expected, output == expected)
                )
            except Exception as e:
                results.append(
                    (inputs, str(e), expected, False)
                )
        return results
    except Exception as e:
        return [
            ("Function Compilation Error", str(e), None, False)
        ]
def main():

    description = (
        "Write a function named n_nat_numbers "
        "that returns numbers from 0 to a given number as a list."
    )
    print("  Natural Language Description:\n", description)
    function_code = generate_function(description)
    print("\n===== Generated Function =====\n")
    print(function_code)
    function_name = "n_nat_numbers"
    test_cases = [
        ((0,), [0]),
        ((1,), [0, 1]),
        ((3,), [0, 1, 2, 3]),
        ((5,), [0, 1, 2, 3, 4, 5]),
        ((10,), [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
    ]
    print("\n===== Executing Tests =====")
```

```
results = execute_function(
        function_code,
        function_name,
        test_cases
    )

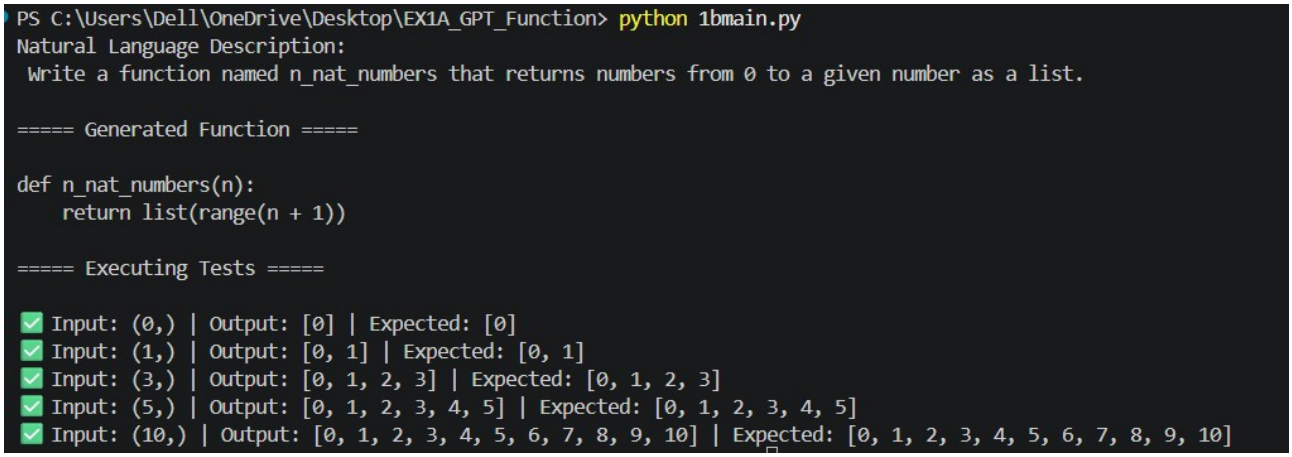
    for inputs, output, expected, passed in results:

        status = "  " if passed else "+"

        print(
            f"{status} Input: {inputs} | "
            f"Output: {output} | "
            f"Expected: {expected}"
        )

if _name___ == "_main_":
    main()
```

**Output Screenshot:**




**Link:**

https://drive.google.com/file/d/1U8sMI4A0SWMU1kbDMycVQzmIbAZyBMRV/view?usp=sharing

**Result:**

Thus, the Python function was successfully generated using Ollama and all the test cases were executed successfully.